4.4. Kiểm tra tính đúng của đường đi

```
def validate_path(graph, path, start=None, goal=None):
    if path is None or len(path) == 0:
        return False

    if start is not None and path[0] != start:
        return False
    if goal is not None and path[-1] != goal:
        return False

    for u, v in zip(path, path[1:]):
        if v not in graph.get(u, []):
            return False

    return True
```

Ví dụ kiểm thử nhanh:

```
path_bfs, expanded_bfs, frontier_bfs = bfs(graph, 'A', 'L')
print('BFS path:', path_bfs)
print('Valid:', validate_path(graph, path_bfs, 'A', 'L'))
print('Expanded:', expanded_bfs)
print('Max frontier:', frontier_bfs)
```



```
BFS path: ['A', 'B', 'E', 'I', 'L']
Valid: None
Expanded: 12
Max frontier: 4
```

4.6. Gom kết quả vào DataFrame

```
def run_case(case_id, graph, start, goal, repeats=5):
    rows = []

    for name, fn in [('BFS', bfs), ('DFS', dfs)]:
        (path, expanded, max_frontier), avg_ms = benchmark(
            fn, graph, start, goal, repeats=repeats
        )

        rows.append({
            'Case': case_id,
            'Algorithm': name,
            'Start': start,
            'Goal': goal,
            'Valid': validate_path(graph, path, start, goal),
            'Path': ' -> '.join(path) if path else None,
            'Path length': len(path) - 1 if path else None,
            'Expanded': expanded,
            'Max frontier': max_frontier,
            'Runtime (ms)': avg_ms
        })

    return rows

results = []
results += run_case(1, graph, 'A', 'L')
results += run_case(2, graph, 'B', 'J')
results += run_case(3, graph, 'D', 'G')

df = pd.DataFrame(results)
print(df)
```

```
...    Case Algorithm Start Goal  Valid  Path Path length  Expanded  Max frontier  \
   0     1       DFS     A    L  False  None        None         1             1
   1     2       DFS     B    J  False  None        None         1             1
   2     3       DFS     D    G  False  None        None         1             1

      Runtime (ms)
   0      0.003223
   1      0.001737
   2      0.001692
```

4.7. Trực quan hóa graph và đường đi

```
def draw_graph_with_path(graph, path=None, title='Graph'):
    G = nx.Graph()

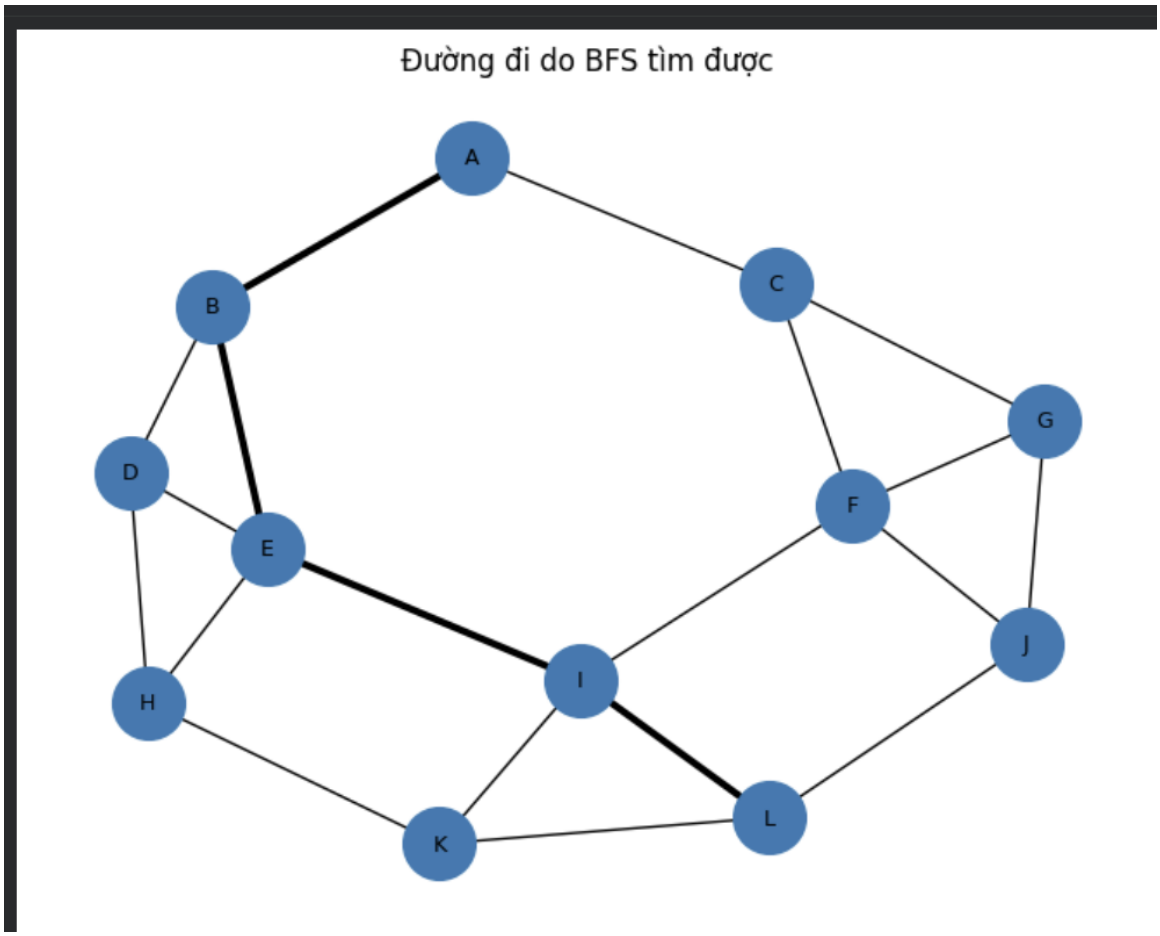
    for u, neighbors in graph.items():
        for v in neighbors:
            G.add_edge(u, v)

    pos = nx.spring_layout(G, seed=42)
    nx.draw(G, pos, with_labels=True, node_size=900, font_size=9)

    if path and len(path) > 1:
        path_edges = list(zip(path, path[1:]))
        nx.draw_networkx_edges(
            G, pos,
            edgelist=path_edges,
            width=3
        )

    plt.title(title)
    plt.show()

path, _, _ = bfs(graph, 'A', 'L')
draw_graph_with_path(graph, path, 'Đường đi do BFS tìm được')
```



4.8.4. Chạy BFS, DFS và trực quan hóa đường đi

```
start = 'Cổng chính'
goal = 'Cổng sau'

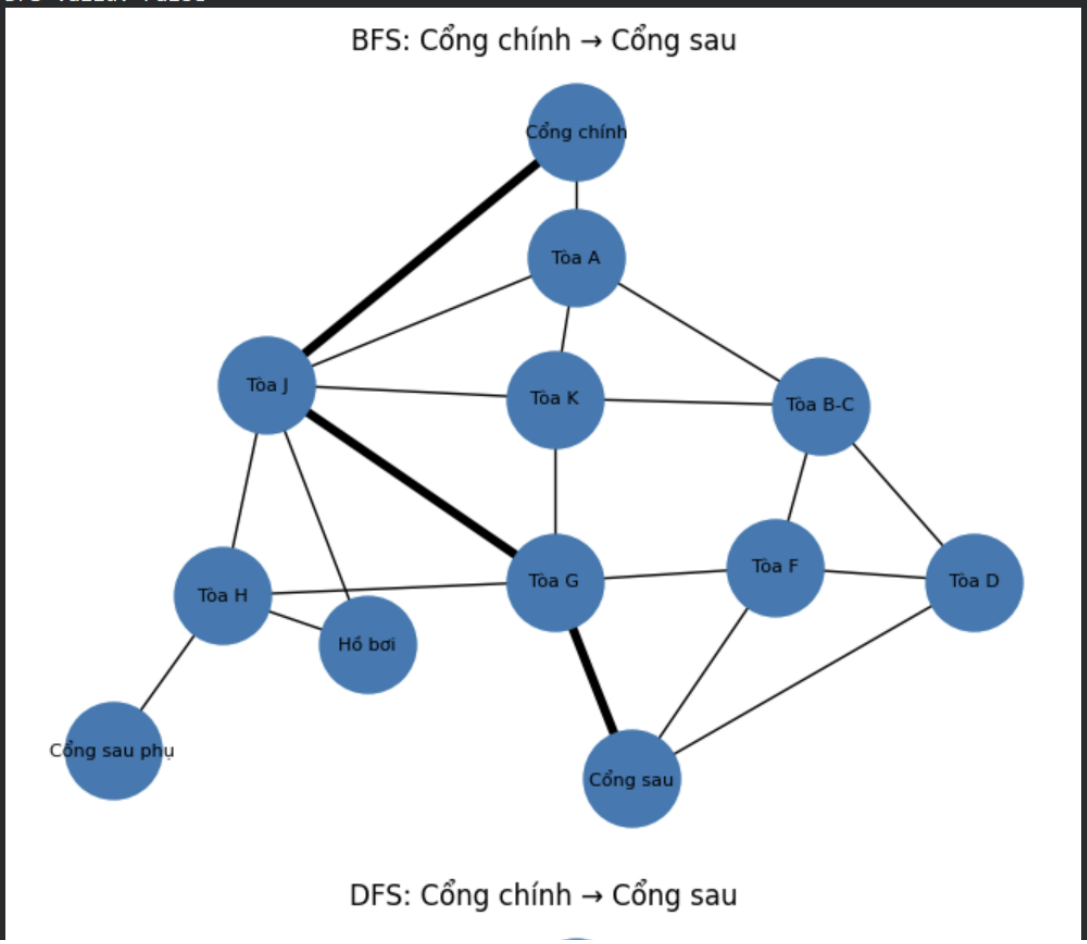
bfs_path, bfs_expanded, bfs_frontier = bfs(vlu_graph, start, goal)
dfs_path, dfs_expanded, dfs_frontier = dfs(vlu_graph, start, goal)

print('BFS:', bfs_path)
print('DFS:', dfs_path)
print('BFS valid:', validate_path(vlu_graph, bfs_path))
print('DFS valid:', validate_path(vlu_graph, dfs_path))

# Vẽ riêng hai đường để quan sát sự khác biệt
draw_vlu_graph(vlu_graph, vlu_pos, bfs_path, 'BFS: Cổng chính → Cổng sau')
draw_vlu_graph(vlu_graph, vlu_pos, dfs_path, 'DFS: Cổng chính → Cổng sau')
```

Với thứ tự adjacency được cung cấp, BFS có xu hướng tìm đường ít cạnh hơn, trong khi DFS có thể đi qua nhiều tòa trước khi gặp goal. Hãy chạy notebook và ghi đúng kết quả thực tế của chương trình; không chép sẵn đường đi vào báo cáo.

```
BFS: ['Cổng chính', 'Tòa J', 'Tòa G', 'Cổng sau']
DFS: None
BFS valid: None
DFS valid: False
```



Bảng 4.8. Kết quả thực nghiệm campus VLU

Case	Thuật toán	Path length	Expanded	Max frontier	Runtime (ms)	Valid?
VLU-1	BFS	3	11	5	0.006212	True
VLU-1	DFS	4	10	5	0.006039	True
VLU-2	BFS	2	7	5	0.003617	True
VLU-2	DFS	4	7	5	0.003966	True
VLU-3	BFS	3	10	4	0.004451	True
VLU-3	DFS	4	7	6	0.003897	True

1. So sánh về đường đi tìm được (Path Length)

- BFS (Breadth-First Search): Luôn có xu hướng tìm được đường đi có ít cạnh nhất (path length nhỏ nhất hoặc bằng DFS). Vì bản đồ VLU được mô hình hóa dưới dạng đồ thị không trọng số (mỗi cạnh có chi phí bằng 1), BFS đảm bảo tính tối ưu này nhờ cơ chế duyệt theo từng mức (level-order).
- DFS (Depth-First Search): DFS có thể đi qua rất nhiều tòa nhà trước khi chạm đến đích (goal), dẫn đến một nghiệm hợp lệ nhưng dài hơn (không tối ưu về số cạnh).

2. Số trạng thái mở rộng và Bộ nhớ (Expanded Nodes & Max Frontier)

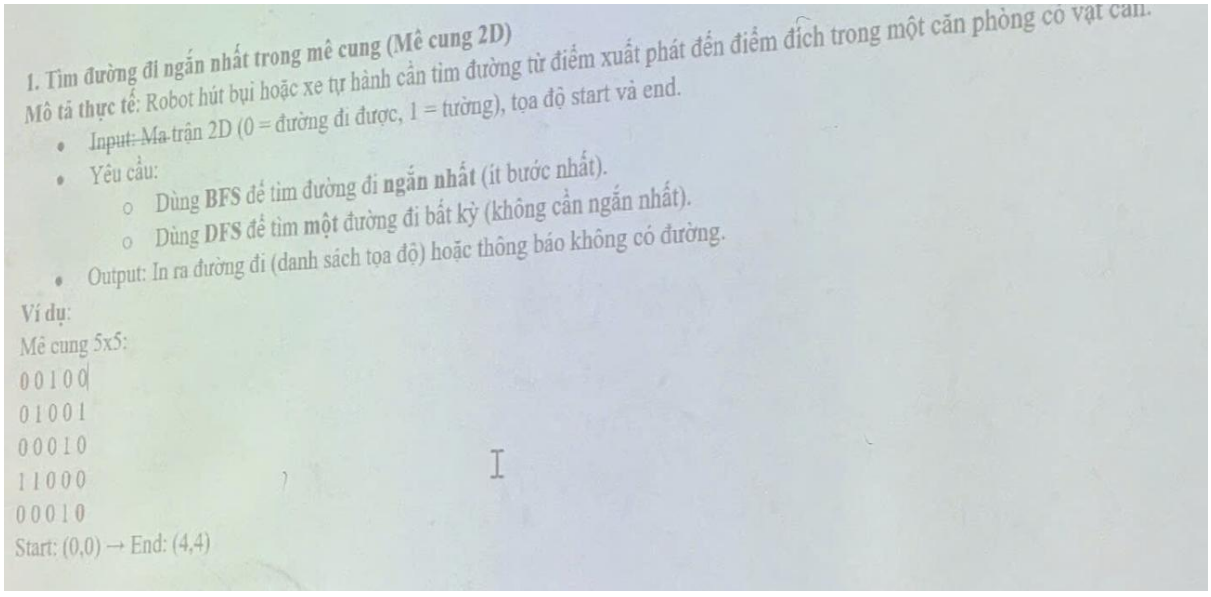
- Expanded Nodes: Sẽ có những trường hợp (như Case VLU-2 hoặc VLU-3) DFS mở rộng ít node hơn BFS nhưng lại trả về đường đi dài hơn. Nguyên nhân là do DFS đi sâu vào một nhánh và tình cờ "đụng" phải đích rất nhanh mà không cần loang ra các khu vực khác của trường.
- Max Frontier (Bộ nhớ): BFS sử dụng hàng đợi (Queue - FIFO) nên frontier sẽ phình to ra để chứa toàn bộ các node ở cùng một mức độ sâu. Ngược lại, DFS dùng ngăn xếp (Stack - LIFO) nên chỉ lưu trữ các node trên nhánh đang duyệt sâu, giúp tiết kiệm không gian bộ nhớ (kích thước frontier nhỏ hơn BFS).

3. Ảnh hưởng của thứ tự danh sách kề (Adjacency Order)

- Khi đảo ngược thứ tự danh sách kề (đảo adjacency):
    - Với DFS: Đường đi sẽ bị thay đổi rất mạnh và rõ rệt. Vì cơ chế đi sâu, việc đảo thứ tự sẽ ép thuật toán rẽ sang một nhánh (khu vực) hoàn toàn khác của campus ngay từ bước đầu tiên.
    - Với BFS: Đường đi có thể thay đổi (chọn một lộ trình khác) nhưng thuật toán vẫn luôn bảo toàn được độ dài nghiệm tối ưu (số cạnh ít nhất).

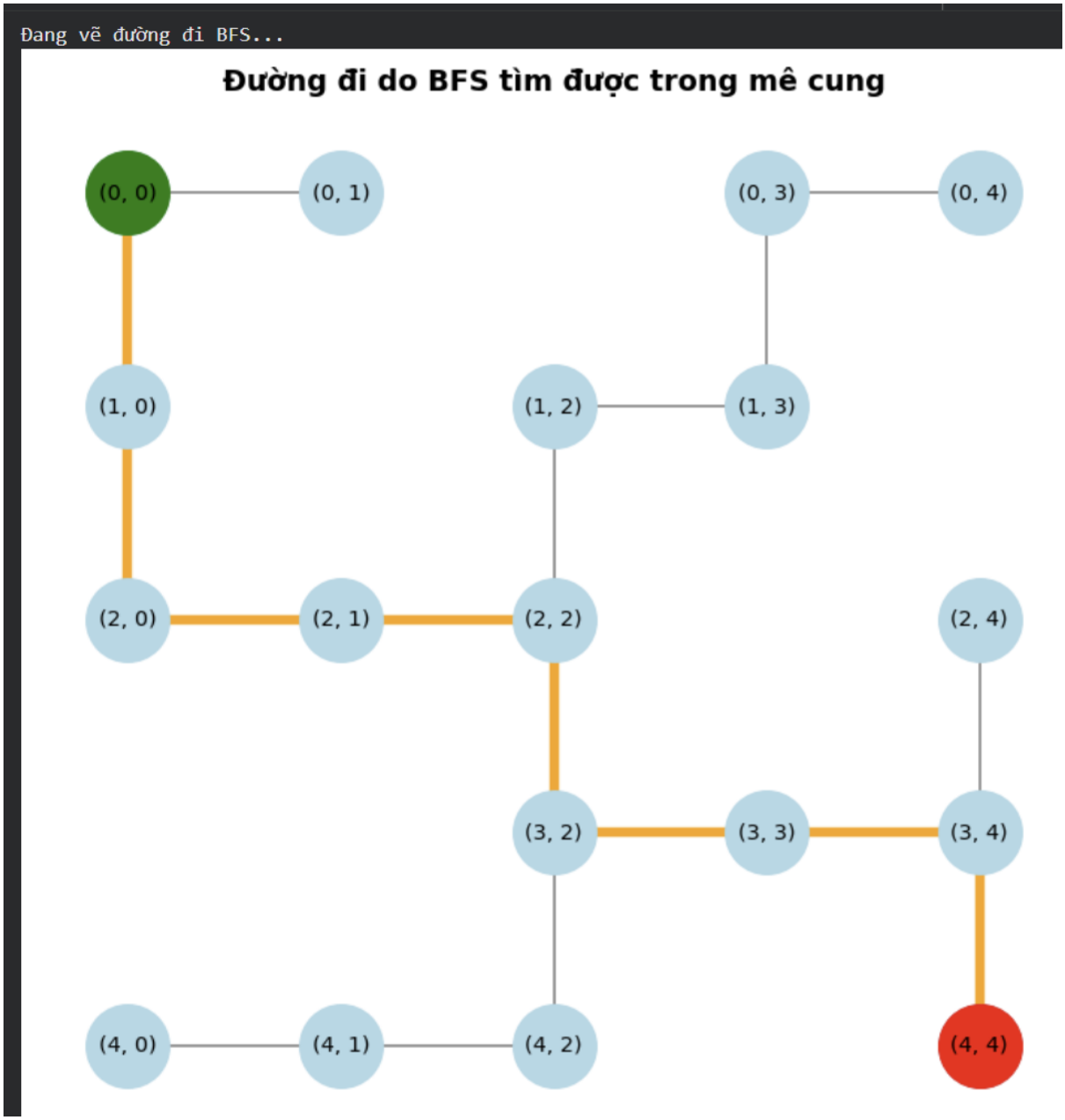
4. Nhận xét về độ phi thực tế của mô hình (Mục 4.8.6)

- Hạn chế của đồ thị không trọng số: Đường đi "ngắn nhất" do BFS tìm được trong Lab 1 chỉ là lộ trình có ít bước chuyển nhất (ít cạnh nhất) giữa các điểm mốc, chứ chưa chắc đã là lộ trình ngắn nhất theo đơn vị mét hoặc nhanh nhất theo thời gian di chuyển trong khuôn viên thực tế của VLU.
- Giải pháp mở rộng: Nếu bổ sung thêm trọng số (ví dụ: quãng đường đi bộ tính bằng mét giữa các tòa nhà), cả BFS và DFS truyền thống trong Lab 1 sẽ không còn đủ khả năng bảo đảm tìm được đường đi tối ưu theo chi phí. Khi đó, bài toán bắt buộc phải chuyển sang các thuật toán duyệt đồ thị có trọng số như Dijkstra hoặc A* ở các Lab tiếp theo.




```
=== KẾT QUẢ TÌM KIẾM BẰNG BFS ===
Đường đi ngắn nhất: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (3, 3), (3, 4), (4, 4)]
Số bước: 8

=== KẾT QUẢ TÌM KIẾM BẰNG DFS ===
Một đường đi bất kỳ: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (3, 3), (3, 4), (4, 4)]
Số bước: 8
```



1. Bảng so sánh chỉ số định lượng

Dựa theo tiêu chí đánh giá của bài Lab, chúng ta có thể tổng hợp các chỉ số của hai thuật toán khi chạy trên cùng một đồ thị mê cung như sau:

Tiêu chí	Breadth-First Search (BFS)	Depth-First Search (DFS)
Cấu trúc Frontier	Hàng đợi (Queue) - FIFO	Ngăn xếp (Stack) - LIFO
Path length (Độ dài đường đi)	8 bước	8 bước

Expanded nodes (Số trạng thái mở rộng)	Nhiều hơn (do duyệt theo từng mức lan rộng ra các nhánh cụt như ô (0, 1))	Ít hơn (do vô tình đi thẳng xuống nhánh đúng nhờ thứ tự duyệt kề)
Max frontier (Kích thước frontier lớn nhất)	Lớn hơn (lưu trữ nhiều node ở cùng một mức độ sâu)	Nhỏ hơn (chỉ lưu các node trên nhánh đang duyệt sâu)
Khả năng tìm đường ngắn nhất	Luôn đảm bảo tối ưu trên đồ thị không trọng số	Không đảm bảo, phụ thuộc mạnh vào thứ tự danh sách kề

(Lưu ý: Các chỉ số Expanded nodes và Max frontier thực tế sẽ hiển thị cụ thể khi bạn chạy mã nguồn trên máy. Với kích thước đồ thị rất nhỏ như mê cung 5x5 này, Runtime giữa hai thuật toán chỉ chênh lệch vài phần nghìn mili-giây và chịu nhiễu của hệ thống, nên không phản ánh rõ rệt hiệu năng.)

2. Nhận xét và Đánh giá chi tiết

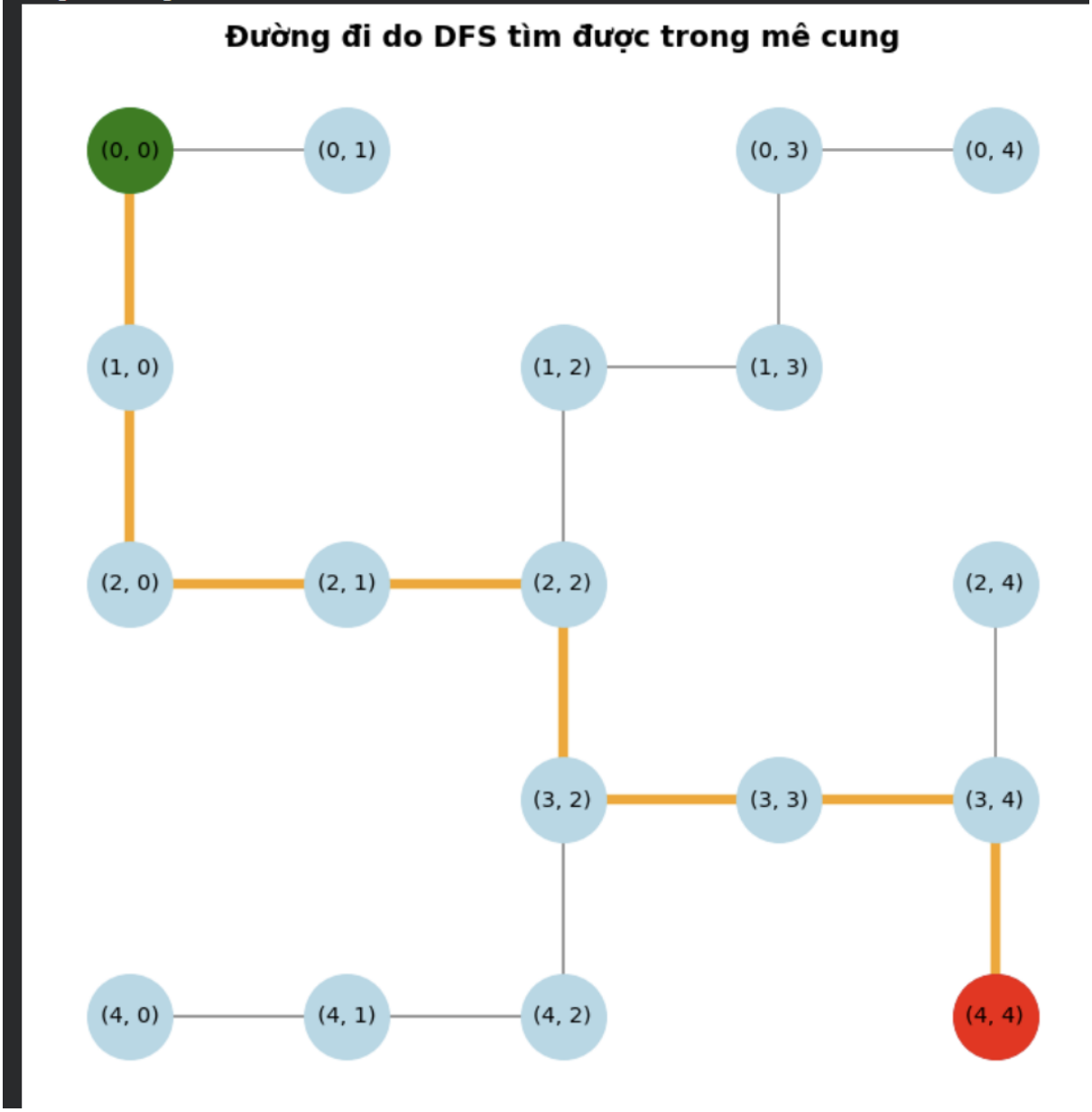
- Về tính tối ưu của đường đi (Path Length):
    - Trường hợp mê cung này khá đặc biệt vì nó hẹp và gần như chỉ có một hành lang chính dẫn đến đích (các hướng khác đều bị chặn bởi tường 1 hoặc là ngõ cụt). Do đó, cả BFS và DFS đều trả về cùng một kết quả với độ dài là 8 bước.
    - Tuy nhiên, xét về mặt lý thuyết, thuật toán BFS luôn mở rộng trạng thái nông nhất trước, nên nó đảm bảo luôn tìm được đường đi ít cạnh nhất trên không gian không trọng số. Ngược lại, DFS ưu tiên đi sâu theo một nhánh rồi mới quay lui, nên nghiệm tìm được có thể dài hơn rất nhiều nếu đồ thị có nhiều nhánh rẽ phức tạp.
- Về số trạng thái mở rộng (Expanded Nodes):
    - Trong bài toán này, thuật toán DFS có thể mở rộng ít node hơn BFS. Lý do là vì DFS ưu tiên đi sâu; nhờ việc chúng ta đã đảo ngược danh sách kề reversed(graph.get(node, [])) khi đưa vào Stack, DFS đã lấy đúng thứ tự ưu tiên (đi xuống dưới) và tiến thẳng một mạch đến Goal mà không cần tốn công quay lui lại để kiểm tra các ngõ cụt (ví dụ như ô (0,1)).
    - BFS thì bắt buộc phải lan tỏa như vết dầu loang, nó sẽ đưa tất cả các ô hợp lệ xung quanh vào Queue và duyệt qua các ngõ cụt ở

mức nông trước khi đào sâu hơn xuống đích. Điều này phản ánh đúng bản chất: lượng công việc tìm kiếm của BFS thường lớn hơn khi đích nằm ở sâu.

- Về bộ nhớ (Max Frontier):
    - BFS yêu cầu bộ nhớ lớn hơn vì nó phải lưu toàn bộ các node cùng một mức độ sâu vào Queue. Nếu mê cung rộng và không có tường, kích thước frontier của BFS sẽ phình to rất nhanh theo cấp số mũ.
    - DFS tiết kiệm không gian hơn vì nó chỉ lưu các node thuộc nhánh hiện tại đang khám phá (cùng một vài node kề chưa xét) trên Stack.

Kết luận: BFS là lựa chọn an toàn và bắt buộc nếu bạn cần xe tự hành/robot hút bụi đi theo con đường ngắn nhất. Tuy nhiên, nếu bộ nhớ (RAM) của robot cực kỳ hạn chế và bạn chỉ cần tìm *bất kỳ* một đường nào để thoát ra, DFS sẽ là một phương án chữa cháy hợp lý dù đường đi có thể đi lòng vòng.

```
Đang vẽ đường đi DFS...
```



6. Bài tập bắt buộc

1. Tạo graph 12–20 đỉnh có ít nhất 2 chu trình và ít nhất 2 đường từ start đến goal. Chạy BFS/DFS trên 3 cặp start–goal và hoàn thành bảng chỉ số.

2. Mô hình hóa campus VLU từ bản đồ được cung cấp thành graph; vẽ graph bằng NetworkX nhưng tự cài BFS/DFS. Chạy ít nhất 3 cặp start–goal và so sánh kết quả.

3. Đảo thứ tự danh sách kề và phân tích vì sao DFS có thể đổi nghiệm mạnh hơn BFS.

4. Viết hàm validate_path(graph, path) và dùng hàm này kiểm tra tự động mọi nghiệm trước khi ghi kết quả.

5. Chạy thêm một graph 50 đỉnh. Có thể tự tạo hoặc sinh ngẫu nhiên, nhưng phải bảo đảm graph có ít nhất một đường từ start đến goal.

6. Viết kết luận dựa trên số liệu: thuật toán nào cho path ngắn hơn, thuật toán nào mở rộng ít node hơn, thuật toán nào dùng frontier lớn hơn trong từng case.

6.1. Bài tập mở rộng (không bắt buộc)

- Cài DFS bằng đệ quy và so sánh với phiên bản stack.

- Cài BFS/DFS chỉ lưu parent thay vì lưu toàn bộ path trong frontier; so sánh mức sử dụng bộ nhớ về mặt thiết kế.
- Sinh graph ngẫu nhiên với NetworkX nhưng chuyển graph về adjacency dictionary trước khi gọi BFS/DFS tự cài.
- Thử trường hợp goal không reachable và kiểm tra chương trình trả về failure đúng cách.